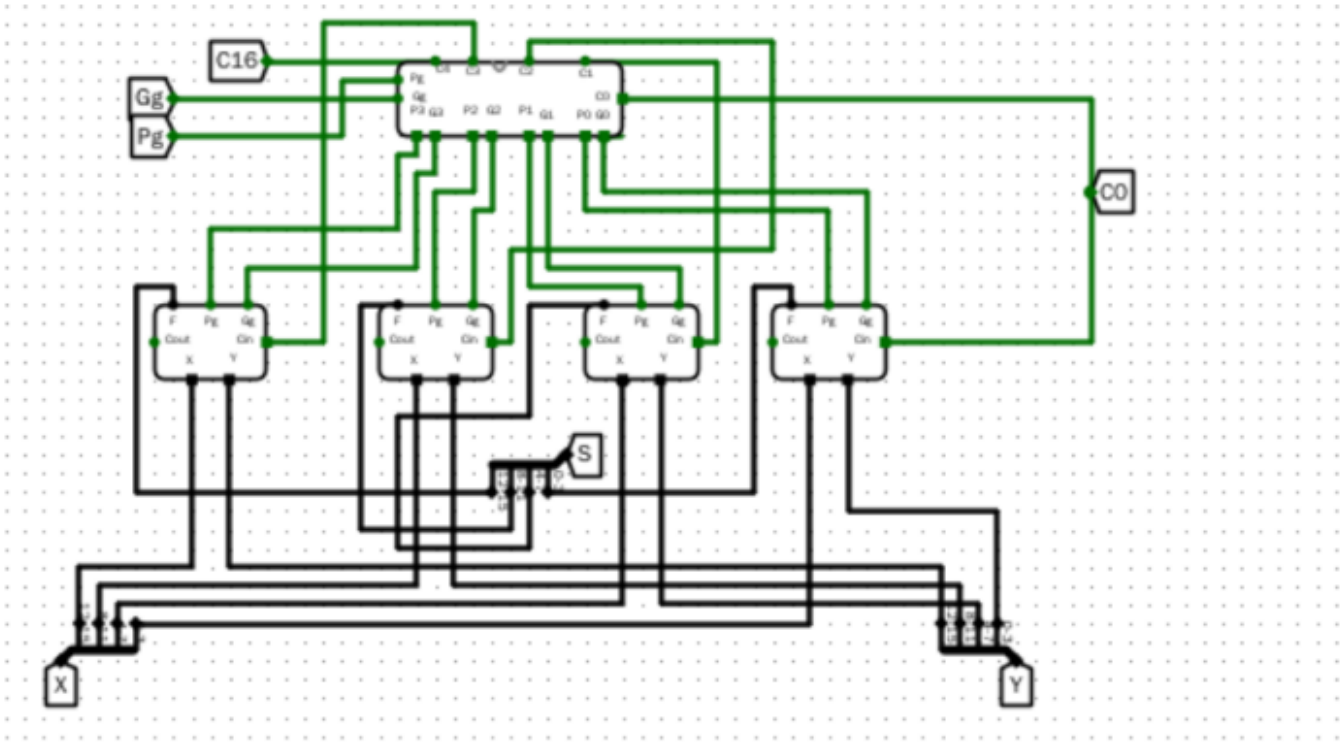
实验报告

思考题

1.通过查找资料，实现一种 64 位二进制快速加法器的设计。

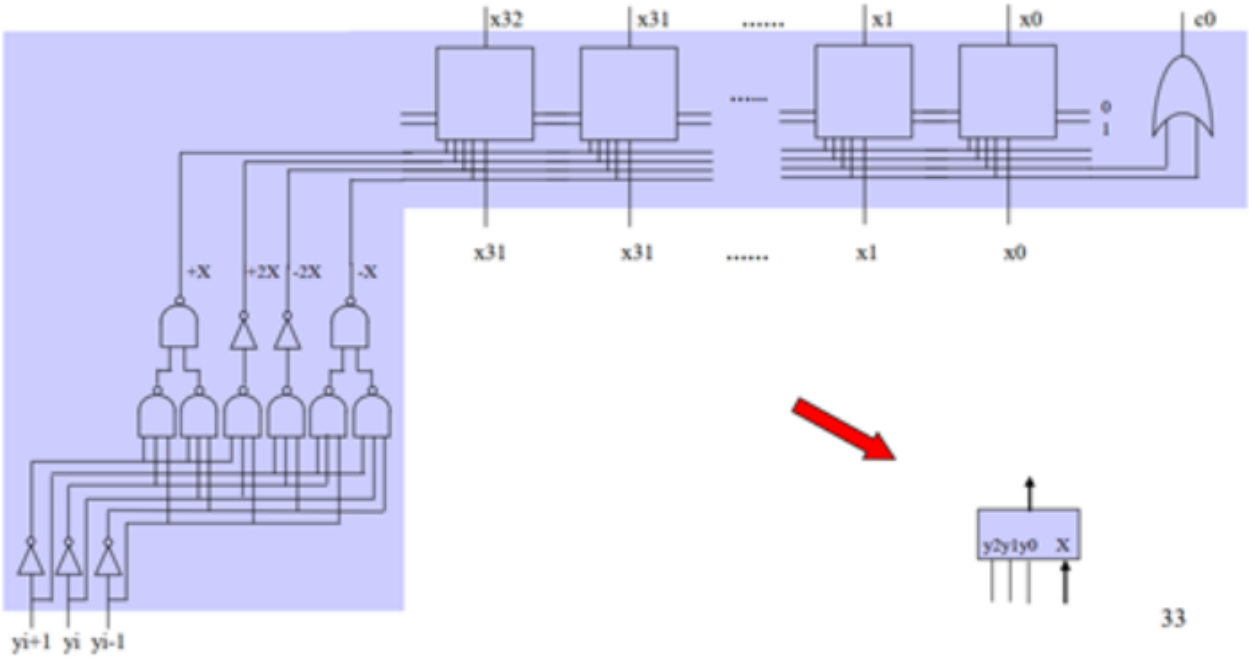
通过四个16位加法器级联实现64位快速加法运算，其原理类似于四个4位加法器组合成16位加法器。由于Logisim仿真工具未直接提供64位输入/输出引脚，可采用高低位拆分方案，即使用两个32位引脚分别处理高32位和低32位数据，并按照相同原理构建电路。具体实现方式可参考类似结构的电路图。



如图所示电路图与该题目原理一致

2.将实验 3 中的快速乘法器设计电路扩展到 32 位无符号数相乘，并探讨如何将该乘法器融合到实验中的 ALU 电路来实现乘法运算。

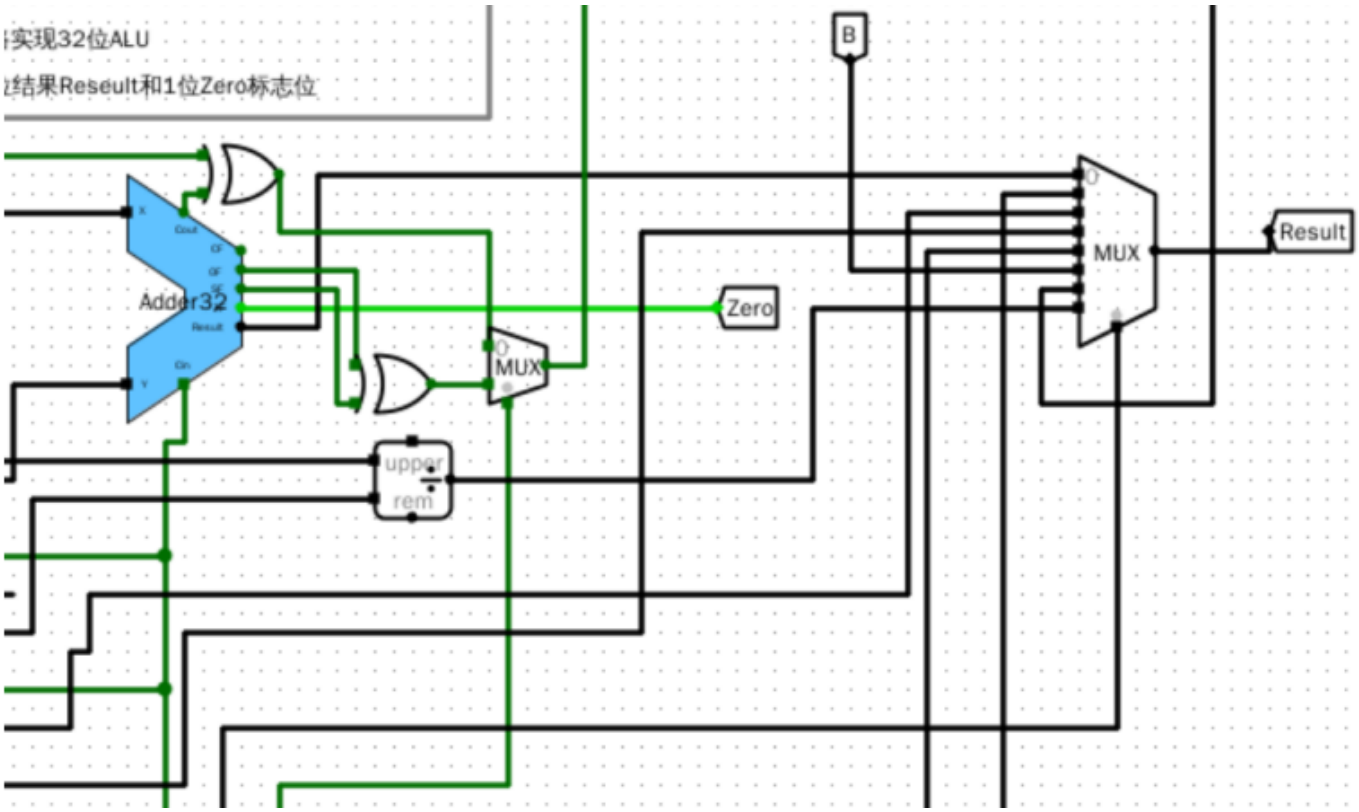
如下图所示实现32位无符号快速加法器，该电路为组合逻辑电路，可以和ALU中的其他功能一样，通过增加多路选择器的选择位数据宽度增加通路数，将该电路整合到ALU中。



32位快速乘法器原理图

3.假设在 RV32I 中新增一条指令，导致在 ALU 中增加一个新运算操作，试修改 ALU 设计电路，并通过测试数据进行验证。

类似于前面乘法加入 ALU 的操作，首先我需要写一个新运算的封装子电路，比如除法，然后将结果连接到 ALU 对应多路选择器剩余的端口，然后对应的在 ALUctrl 的 Optcr 中添加对应的编码序号，如果 Opctr 位数不够或者多路选择器端口不够可以拓展。即可实现。



将除法功能添加到ALU中

4.分析比较运算使用独立的比较器和使用减法运算通过标志位来实现两种方法的特性

由于独立比较器是组合逻辑实现，而减法加标志位需要设计算术运算的电路，所以在原理上前者比较简单，并且因为是组合逻辑所以独立比较器运行较快，而相对而言减法标志位判断较慢，但是独立比较器硬件开销比较大，对于每一种可能情况需要设计对应的逻辑电路，跟真值表原理类似，而在这方面减法标志位的方法开销会小很多，并且前者只能实现比较，而后者更加灵活，还能实现减法等操作，可以复用，综上所述为二者特性。

基本信息

实验名称 运算部件设计 **实验人：**241220071葛家韬

实验目的

1. 掌握先行进位部件 CLU 和先行进位加法器 CLA 的设计方法。
2. 掌握 32 位快速加法器和标志位的设计方法。
3. 掌握 32 位桶形移位器的设计方法。
4. 掌握 RV32I 算术逻辑部件的设计方法。

实验环境

Logisim 2.16

实验内容

1. 4位先行进位加法器 CLA

（1）实验整体方案设计

> 要求：说明本次实验的顶层设计模块图，对每个子模块进行详细描述，定义输入输出引脚，数据及控制信号的传输通道等。

对于一个 16 位加法器，可以分成 4组，每组用一个 4 位先行进位加法器 CLA 实现。

实验电路较为简单，不需要顶层模块设计图。

（2）实验原理图和电路图

> 要求：给出每个子模块的原理图和 Logisim 中的电路图，定义子模块的外观图。如果对实验指导讲义中的内容提出优化或改进，需要此说明原因、方法和效果。

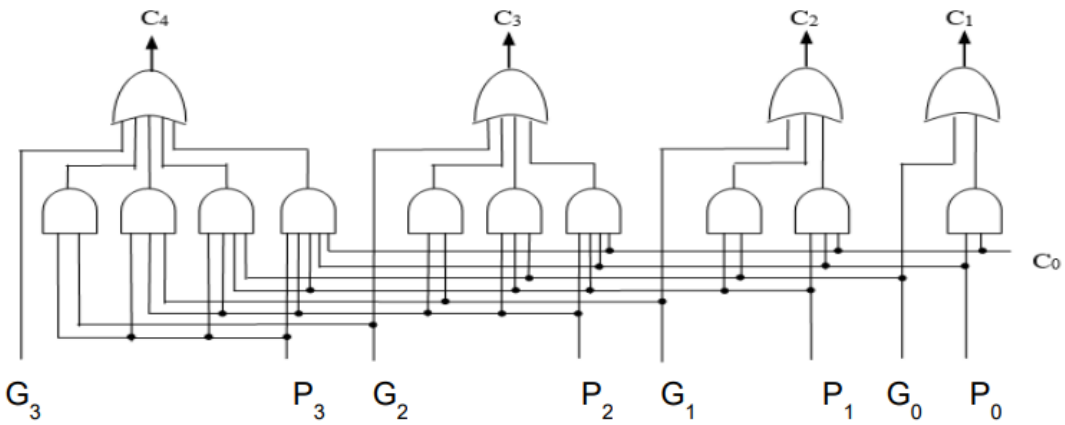


图1.0 原理图

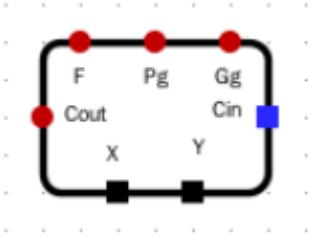


图1.1 封装情况

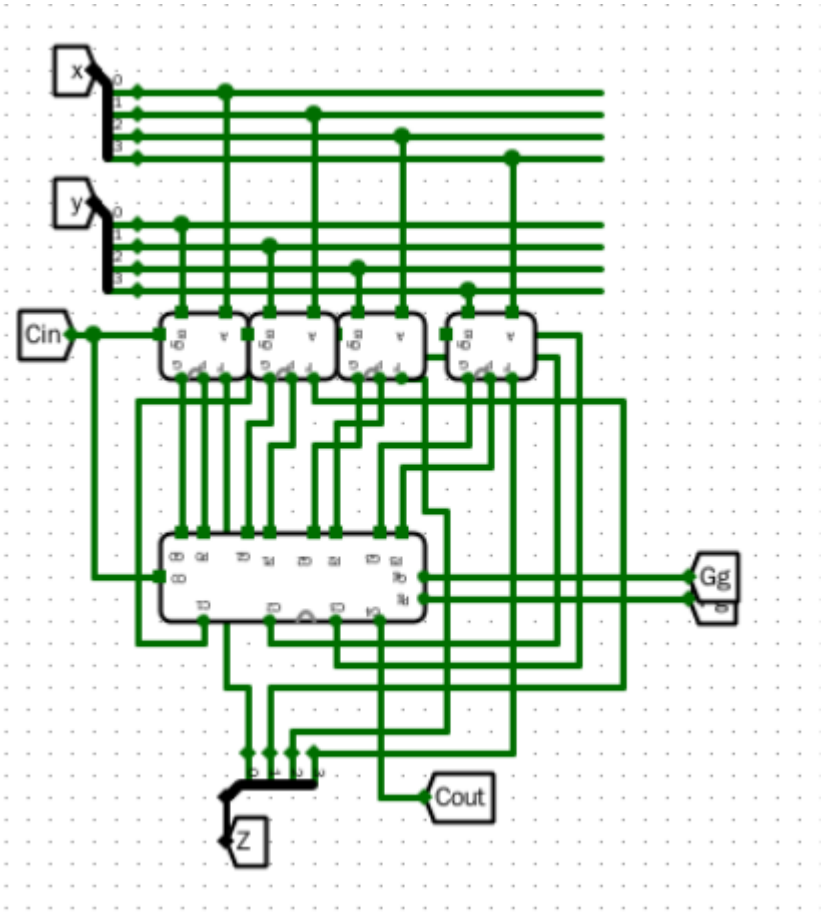


图1.2 电路图

（3）实验数据仿真测试图

> 要求：根据实验要求，输入测试数据，选择单步时钟执行，截取仿真运行时的电路图，分析电路状态是否满足设计需求。说明子模块的功能，列出子模块的功能表。

—— 预期输出 ——

Cnt	X	Y	Cin	Z	Cout	Gg	Pg
00	f	2	0	1	1	1	1
01	e	3	0	1	1	1	1
02	d	4	0	1	1	1	0
03	c	5	0	1	1	1	0
04	b	6	0	1	1	1	1
05	a	7	0	1	1	1	1
06	9	8	0	1	1	1	0
07	8	9	0	1	1	1	0
08	7	a	0	1	1	1	1
09	6	b	0	1	1	1	1
0a	5	c	0	1	1	1	0
0b	4	d	0	1	1	1	0
0c	3	e	0	1	1	1	1
0d	2	f	0	1	1	1	1
0e	1	f	0	0	1	1	1
0f	0	f	0	f	0	0	1
10	f	f	1	f	1	1	1
11	e	f	1	e	1	1	1
12	c	f	1	c	1	1	1
13	a	f	1	a	1	1	1
14	8	7	1	0	1	0	1
15	6	9	1	0	1	0	1
16	4	b	1	0	1	0	1
17	2	d	1	0	1	0	1
18	3	c	1	0	1	0	1
19	5	c	1	2	1	1	0
1a	7	c	1	4	1	1	1
1b	9	c	1	6	1	1	0
1c	b	c	1	8	1	1	1
1d	d	c	1	a	1	1	0

—— 实际输出 ——

展示原始

Cnt	X	Y	Cin	Z	Cout	Gg	Pg
00	f	2	0	1	1	1	1
01	e	3	0	1	1	1	1
02	d	4	0	1	1	1	0
03	c	5	0	1	1	1	0
04	b	6	0	1	1	1	1
05	a	7	0	1	1	1	1
06	9	8	0	1	1	1	0
07	8	9	0	1	1	1	0
08	7	a	0	1	1	1	1
09	6	b	0	1	1	1	1
0a	5	c	0	1	1	1	0
0b	4	d	0	1	1	1	0
0c	3	e	0	1	1	1	1
0d	2	f	0	1	1	1	1
0e	1	f	0	0	1	1	1
0f	0	f	0	f	0	0	1
10	f	f	1	f	1	1	1
11	e	f	1	e	1	1	1
12	c	f	1	c	1	1	1
13	a	f	1	a	1	1	1
14	8	7	1	0	1	0	1
15	6	9	1	0	1	0	1
16	4	b	1	0	1	0	1
17	2	d	1	0	1	0	1
18	3	c	1	0	1	0	1
19	5	c	1	2	1	1	0
1a	7	c	1	4	1	1	1
1b	9	c	1	6	1	1	0
1c	b	c	1	8	1	1	1
1d	d	c	1	a	1	1	0

图1.3 仿真测试图

（4）错误现象及分析

> 要求：在电路设计、连接和仿真运行时，遇到的任何错误，都需要截屏放置到实验报告中，并分析错误原因和解决办法。

在构建CLU部件时，由于对进位生成函数（Gi=XiYi）和进位传递函数（Pi=Xi+Yi）的逻辑关系理解不足，初期错误地将Pi误设为异或运算（Xi⊕Yi），导致进位链计算结果与理论值不符。其次，在扩展为多级CLA时，组间进位函数Pg和Gg的表达式（Pg=P3P2P1P0，Gg=G3+P3G2+P3P2G1+P3P2P1G0）未正确级联，使得高位进位输出C4=Gg+PgC0出现延迟偏差。此外，测试环节中未充分覆盖边界条件（如全1相加时的进位溢出），导致封装后的CLA4电路在极端输入下输出异常。通过逐级对比门级仿真结果与理论真值表，最终发现错误源于或门层级连接顺序错误。

2. 16位两级先行进位加法器实验

（1）实验整体方案设计

> 要求：说明本次实验的顶层设计模块图，对每个子模块进行详细描述，定义输入输出引脚，数据及控制信号的传输通道等。 本实验的核心目标是优化加法器的进位生成机制以提高运算速度，采用先行进位技术（Carry Lookahead）通过CLU部件预先计算进位信息，有效规避了传统加法器逐位进位的串行延迟问题。具体实现中，首先基于进位传递函数Pi=Xi+Yi和进位生成函数Gi=Xi·Yi构建4位加法器的并行进位逻辑表达式（如C1=G0+P0C0），使各进位信号C1-C4仅依赖初始输入即可同步生成，完全消除了进位链的级联延迟。为支持更大位宽运算，实验进一步引入组间并行进位机制，通过定义组间进位生成函数Gg=G3+P3G2+P3P2G1+P3P2P1G0和传递函数Pg=P3P2P1P0，将4位CLU模块分级扩展，实现组内与组

> 间的双重并行进位，最终构建出支持16/32位的高速加法器架构。该设计通过多级超前进位策略，显著降低了大规模加法运算的关键路径延迟。

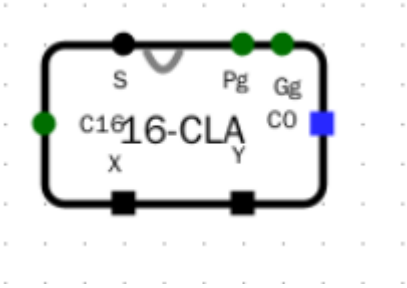


图2.1 电路封装

（2）实验原理图和电路图

> 要求：给出每个子模块的原理图和 Logisim 中的电路图，定义子模块的外观图。如果对实验指导讲义中的内容提出优化或改进，需要此说明原因、方法和效果。

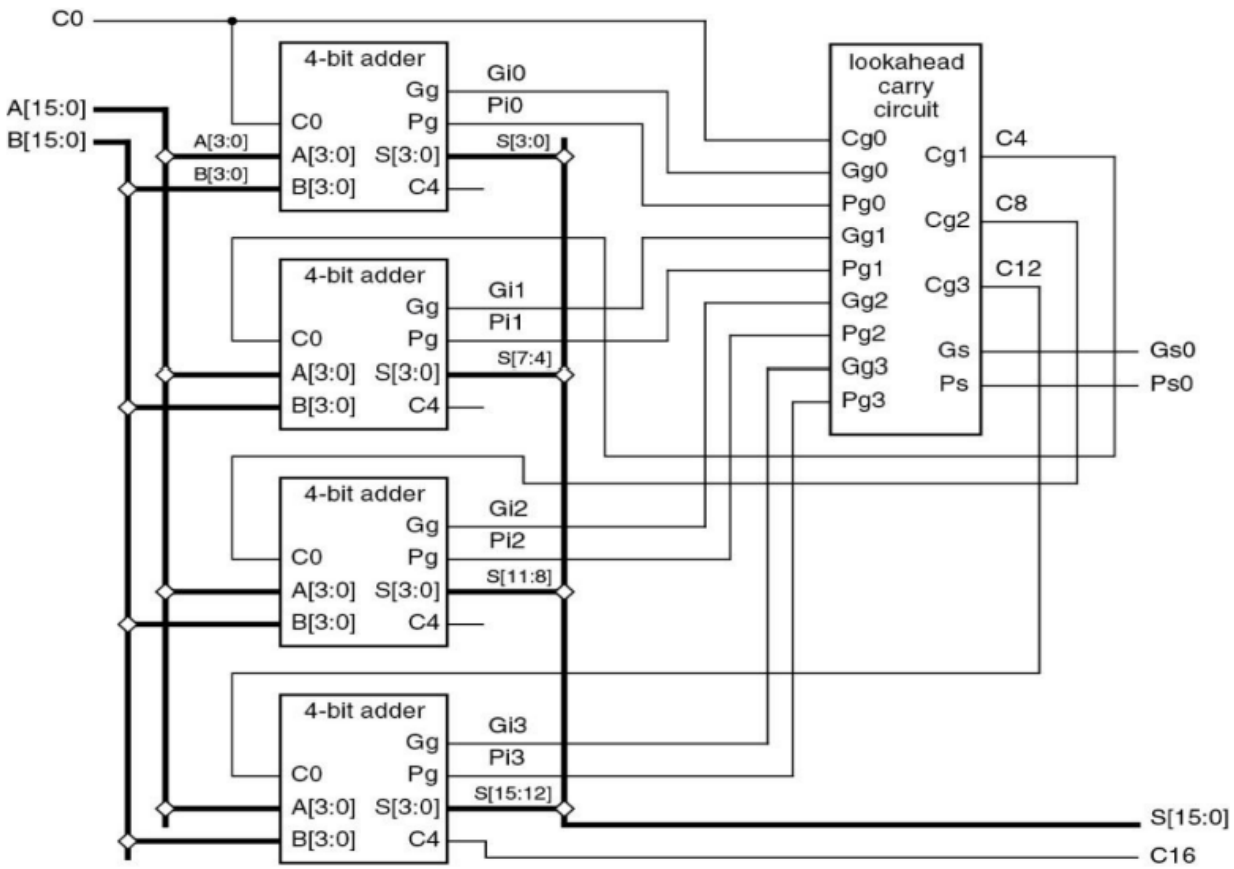


图 4.8　16 位两级先行进位加法器原理图

图2.2 实验原理图

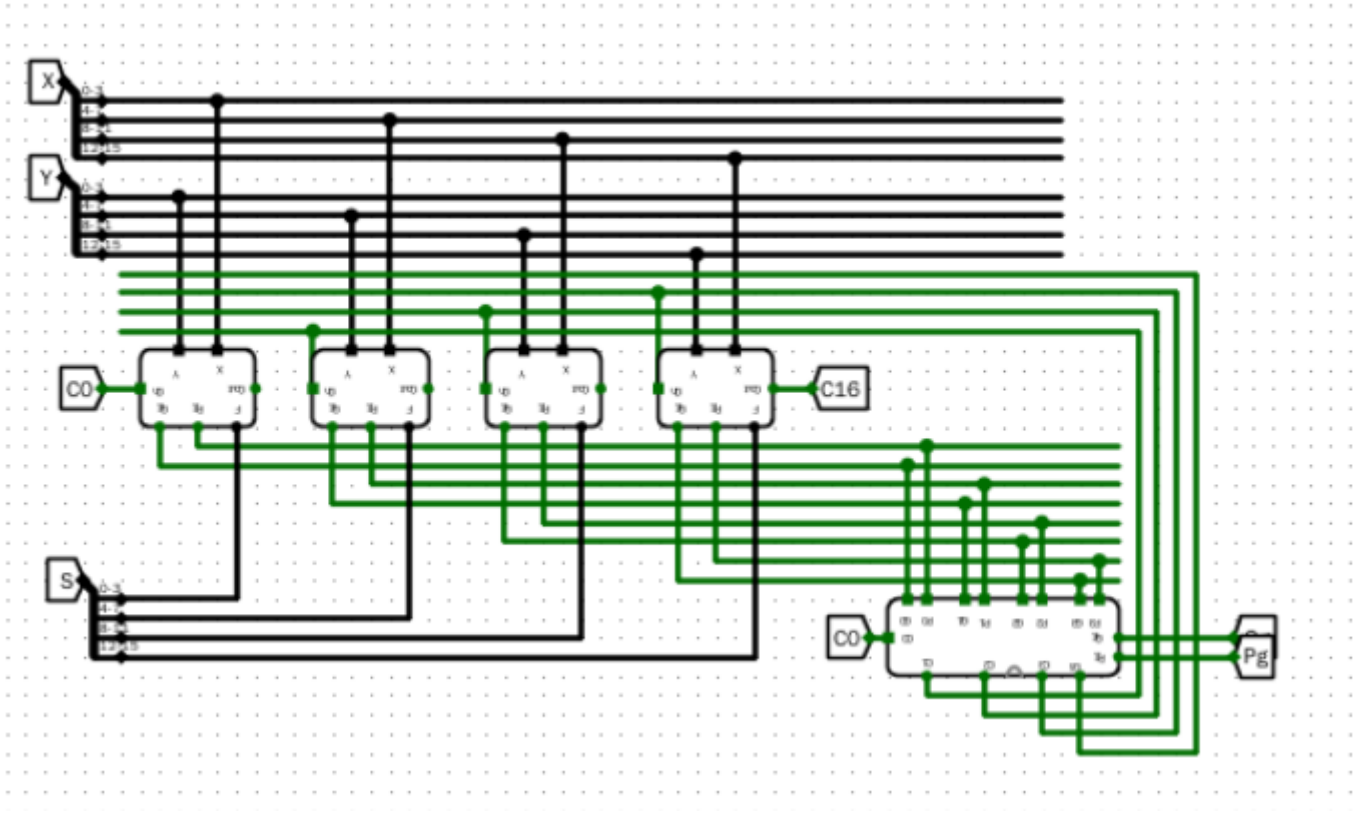
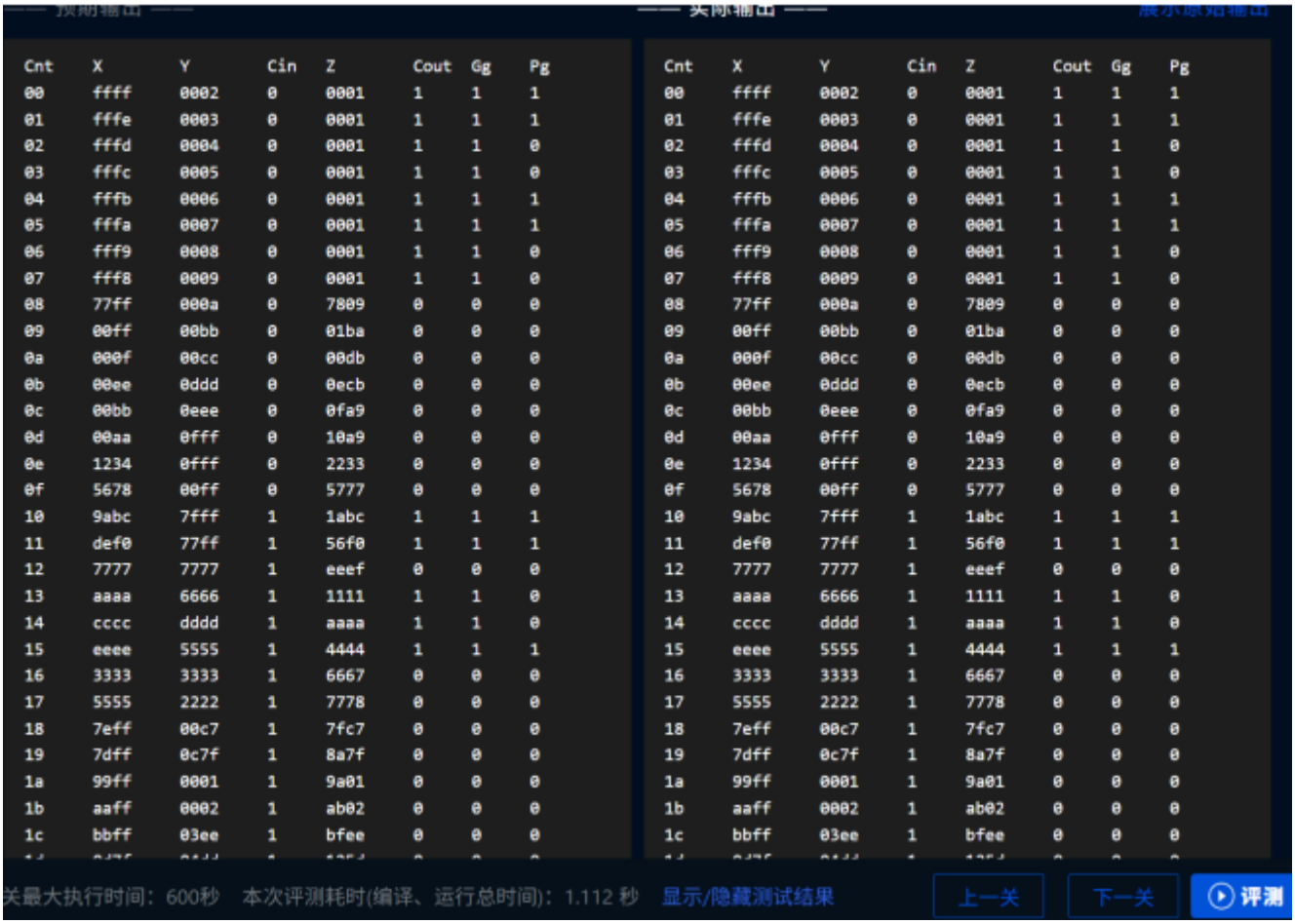


图2.3 电路图

(3)仿真测试



Cnt	X	Y	Cin	Z	Cout	Gg	Pg
00	ffff	0002	0	0001	1	1	1
01	fffe	0003	0	0001	1	1	1
02	fffd	0004	0	0001	1	1	0
03	fffc	0005	0	0001	1	1	0
04	fffb	0006	0	0001	1	1	1
05	fffa	0007	0	0001	1	1	1
06	fff9	0008	0	0001	1	1	0
07	fff8	0009	0	0001	1	1	0
08	77ff	000a	0	7809	0	0	0
09	00ff	00bb	0	01ba	0	0	0
0a	000f	00cc	0	00db	0	0	0
0b	00ee	0ddd	0	0ecb	0	0	0
0c	00bb	0eee	0	0fa9	0	0	0
0d	00aa	0fff	0	10a9	0	0	0
0e	1234	0fff	0	2233	0	0	0
0f	5678	00ff	0	5777	0	0	0
10	9abc	7fff	1	1abc	1	1	1
11	def0	77ff	1	56f0	1	1	1
12	7777	7777	1	eeef	0	0	0
13	aaaa	6666	1	1111	1	1	0
14	cccc	dddd	1	aaaa	1	1	0
15	eeee	5555	1	4444	1	1	1
16	3333	3333	1	6667	0	0	0
17	5555	2222	1	7778	0	0	0
18	7eff	00c7	1	7fc7	0	0	0
19	7dff	0c7f	1	8a7f	0	0	0
1a	99ff	0001	1	9a01	0	0	0
1b	aaff	0002	1	ab02	0	0	0
1c	bbff	03ee	1	bfee	0	0	0

图2.4 电路仿真测试情况

（4）错误现象及分析

> 要求：在电路设计、连接和仿真运行时，遇到的任何错误，都需要截屏放置到实验报告中，并分析错误原因和解决办法。

在16位两级先行进位加法器实验中，可能出现以下关键错误：首先，在组间进位传递上，若未正确连接低4位CLU的进位输出到高位CLU的进位输入，会导致进位链断裂；其次，组内进位生成函数Gg和传递函数Pg的计算若出现逻辑门连接错误（如将或门误接为与门），将导致高位进位结果异常；此外，在封装子电路时，若引脚顺序映射错误（如将第三组的进位输出误接到第四组的数据输入），会造成计算结果系统性偏差；最后，测试阶段若未验证边界条件（如全1相加时的进位溢出），可能掩盖高位进位丢失的问题。这些错误会直接影响加法器的正确性和扩展性，需通过逐级对比真值表和门级仿真来排查。

3. 32 位快速加法器实验

（1）实验整体方案设计

> 要求：说明本次实验的顶层设计模块图，对每个子模块进行详细描述，定义输入输出引脚，数据及控制信号的传输通道等。

本实验通过级联两个16位先行进位加法器（CLA）构建32位快速加法器，并集成标志位生成电路。在Logisim中，首先加载16位CLA模块作为基础组件，通过精确连接高低位加法器的进位链（低位Cout接高位Cin）实现32位运算。标志位电路采用优化的逻辑设计：溢出标志OF通过组合最高位符号位运算（$A_{n-1}B_{n-1}F_{n-1}$ + $A_{n-1}B_{n-1}F_{n-1}$）实现；零标志ZF采用分组级联的或门结构（$F_{n-1}+F_{n-2}+...+F_0$）以避免32位直接级联的扇入问题；符号标志SF直接取最高位输出（F_{n-1}）；进位标志CF由最终进位与初始进位异或（Cout⊕Cin）生成。实验关键点在于确保进位链的正确传递，以及标志位逻辑与理论真值表的一致性，最终封装完成的32位加法器需通过边界测试（如全1相加、符号位溢出等场景）验证功能正确性，并保存为可复用的电路模块。

电路封装如下：

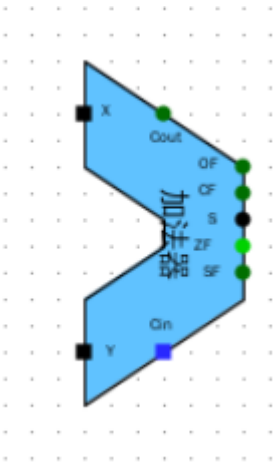


图3.1 封装情况

（2）实验原理图和电路图

> 要求：给出每个子模块的原理图和 Logisim 中的电路图，定义子模块的外观图。如果对实验指导讲义中的内容提出优化或改进，需要此说明原因、方法和效果。

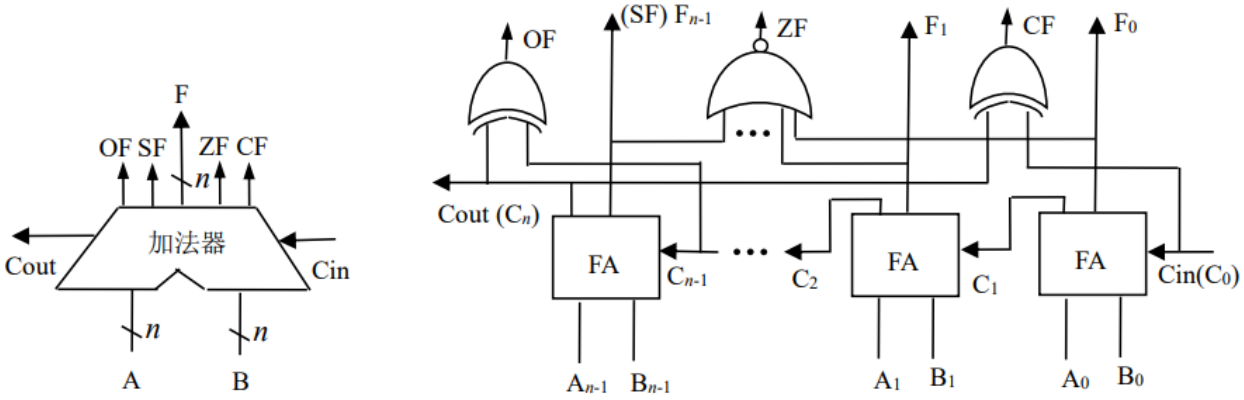


图3.2实验原理图

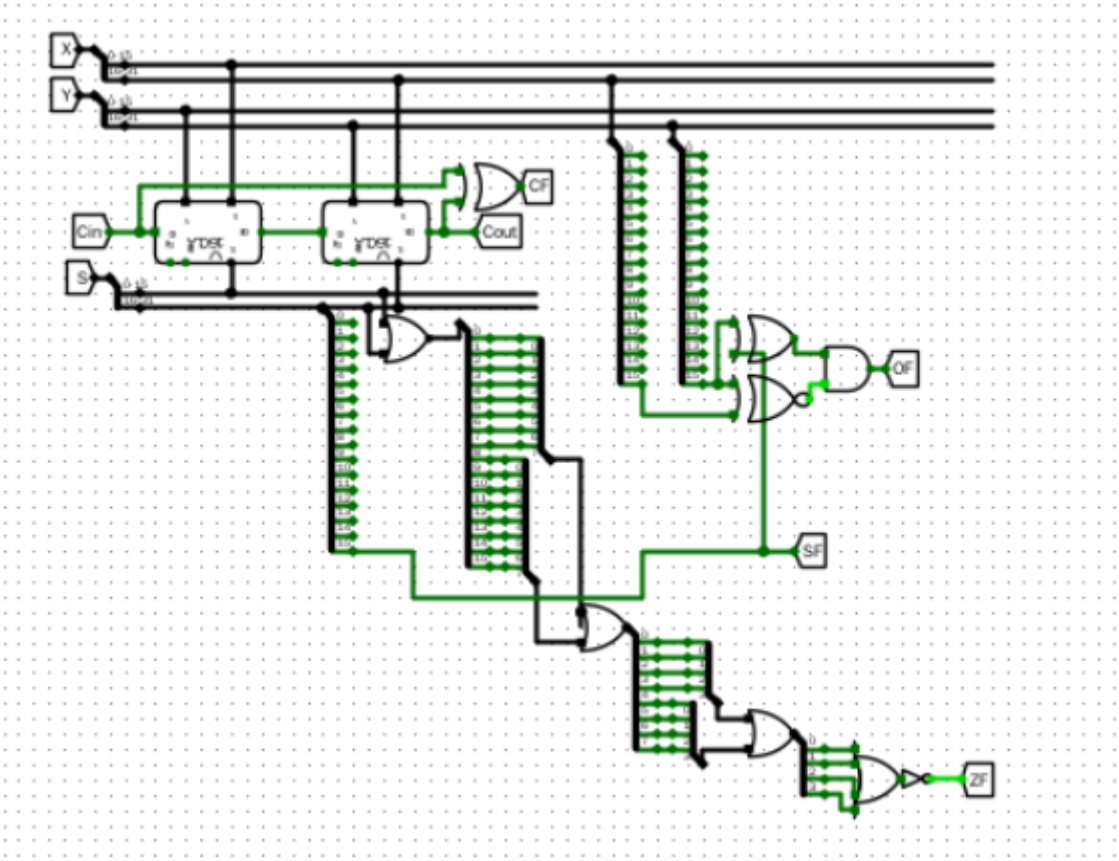


图3.5 电路图

（3）实验数据仿真测试图

> 要求：根据实验要求，输入测试数据，选择单步时钟执行，截取仿真运行时的电路图，分析电路状态是否满足设计需求。说明子模块的功能，列出子模块的功能表。

—— 预期输出 ——

Cnt	X	Y	Cin	S	Cout	OF
00	ffffffff	00000002	0	00000001	1	0
01	fffffffe	80000003	0	80000001	1	0
02	7fffffffd	00000004	0	80000001	0	1
03	fffffffc	90000005	0	90000001	1	0
04	fffffffb	00000006	0	00000001	1	0
05	fffffffa	a0000007	0	a0000001	1	0
06	50000009	76543210	0	c6543219	0	1
07	708090ab	70000009	0	e08090b4	0	1
08	0077ffee	0000000a	0	0077fff8	0	0
09	00ff00ff	000000bb	0	00ff01ba	0	0
0a	12345678	0000ccaa	0	12352322	0	0
0b	77777777	00dddccc	0	78555443	0	0
0c	bbbbbbbb	000eeefd	0	bbcaaab8	0	0
0d	89abcdef	00fff555	0	8aabc344	0	0
0e	12341234	00fff333	0	13340567	0	0
0f	56785678	00ff77dd	0	5777ce55	0	0
10	ffff9abc	00007fff	1	00001abc	1	0
11	ffffdef0	000077ff	1	000056f0	1	0
12	77775555	00007777	1	7777cccd	0	0
13	ffffaaaa	06666111	1	06660bbc	1	0
14	ffffcccc	ffffdddd	1	ffffaaaa	1	0
15	ffffeeee	005555aa	1	00554499	1	0
16	33332222	00003333	1	33335556	0	0
17	5555dddd	002222bb	1	55780099	0	0
18	07effdbe	000000c7	1	07effe86	0	0
19	00007dff	00c7fbce	1	00c879ce	0	0
1a	ffff99ff	00012345	1	0000bd45	1	0
1b	ffffaaff	00234567	1	0022f067	1	0
1c	ffffbbff	03eeddcc	1	03ee99cc	1	0
1d	d7f34567	004dd5ee	1	d8411b56	0	0
1e	7f7f7f7f	00defdef	1	805e7d6f	0	1
1f	ffffdead	ffffdead	1	ffffbd5b	1	0

—— 实际输出 ——　展示原始输出

Cnt	X	Y	Cin	S	Cout	OF
00	ffffffff	00000002	0	00000001	1	0
01	fffffffe	80000003	0	80000001	1	0
02	7fffffffd	00000004	0	80000001	0	1
03	fffffffc	90000005	0	90000001	1	0
04	fffffffb	00000006	0	00000001	1	0
05	fffffffa	a0000007	0	a0000001	1	0
06	50000009	76543210	0	c6543219	0	1
07	708090ab	70000009	0	e08090b4	0	1
08	0077ffee	0000000a	0	0077fff8	0	0
09	00ff00ff	000000bb	0	00ff01ba	0	0
0a	12345678	0000ccaa	0	12352322	0	0
0b	77777777	00dddccc	0	78555443	0	0
0c	bbbbbbbb	000eeefd	0	bbcaaab8	0	0
0d	89abcdef	00fff555	0	8aabc344	0	0
0e	12341234	00fff333	0	13340567	0	0
0f	56785678	00ff77dd	0	5777ce55	0	0
10	ffff9abc	00007fff	1	00001abc	1	0
11	ffffdef0	000077ff	1	000056f0	1	0
12	77775555	00007777	1	7777cccd	0	0
13	ffffaaaa	06666111	1	06660bbc	1	0
14	ffffcccc	ffffdddd	1	ffffaaaa	1	0
15	ffffeeee	005555aa	1	00554499	1	0
16	33332222	00003333	1	33335556	0	0
17	5555dddd	002222bb	1	55780099	0	0
18	07effdbe	000000c7	1	07effe86	0	0
19	00007dff	00c7fbce	1	00c879ce	0	0
1a	ffff99ff	00012345	1	0000bd45	1	0
1b	ffffaaff	00234567	1	0022f067	1	0
1c	ffffbbff	03eeddcc	1	03ee99cc	1	0
1d	d7f34567	004dd5ee	1	d8411b56	0	0
1e	7f7f7f7f	00defdef	1	805e7d6f	0	1
1f	ffffdead	ffffdead	1	ffffbd5b	1	0

图3.7 仿真测试状态图

（4）错误现象及分析

> 要求：在电路设计、连接和仿真运行时，遇到的任何错误，都需要截屏放置到实验报告中，并分析错误原因和解决办法。

在32位快速加法器实验中，可能出现以下关键错误：首先，在级联两个16位加法器时，若未正确处理高低位间的进位传递（如将低位加法器的Cout错误连接到高位加法器的非进位输入端），会导致计算结果出现系统性偏差。其次，在标志位生成电路中，OF标志的实现若错误使用与非门替代原始公式中的与或逻辑（$OF=An-1\cdot Bn-1\cdot Fn-1+An-1\cdot Bn-1\cdot Fn-1$），将无法正确检测溢出；而ZF标志的分组分级计算中，若未合理控制扇入数（如直接级联32个或门），可能因信号竞争导致输出振荡。此外，CF标志的异或逻辑（Cout⊕Cin）若被误接为同或门，会使进位标志极性反转。测试阶段需特别注意边界条件，如全1加法时若未验证CF和ZF的联动关系，可能掩盖高位进位丢失的错误。最后，封装时若错误映射端口顺序（如将SF误接至低位加法器的输出），会导致标志位与计算结果失配。

4.32 位桶形移位器实验

（1）实验整体方案设计

> 要求：说明本次实验的顶层设计模块图，对每个子模块进行详细描述，定义输入输出引脚，数据及控制信号的传输通道等。

本实验在原有8位桶形移位器的基础上进行扩展，构建支持32位数据宽度和5位移位位数的桶形移位器。通过增加两级多路选择器（分别对应8位和16位移位）实现完整的移位功能，其中控制信号扩展至5位以满足32位移位

需求（最大移位31位）。在Logisim实现过程中，重点优化了数据通路设计：采用分层级联的选择器结构，将1/2/4/8/16位移位模块按二进制权重级联，并通过分线器精确控制各段数据选通。关键改进包括：1）32位数据通路的位扩展处理，确保高低位数据对齐；2）算术/逻辑移位模式的统一控制，通过符号位扩展电路实现算术右移；3）移位方向（左/右）的统一控制逻辑。实验验证需特别关注边界情况，如最大位移量（31位）时的数据完整性、符号位扩展正确性等。最终封装完成的Shifter32模块支持所有RV32I指令集要求的移位操作（SLL/SRL/SRA），并通过功能测试后保存为可复用电路组件。该设计显著提升了处理器的移位运算效率，关键路径延迟控制在5个门级以内。

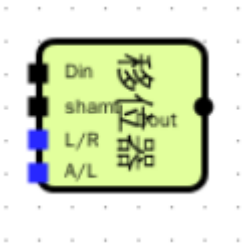


图4.1 电路封装情况

（2）实验原理图和电路图

> 要求：给出每个子模块的原理图和 Logisim 中的电路图，定义子模块的外观图。如果对实验指导讲义中的内容提出优化或改进，需要此说明原因、方法和效果。

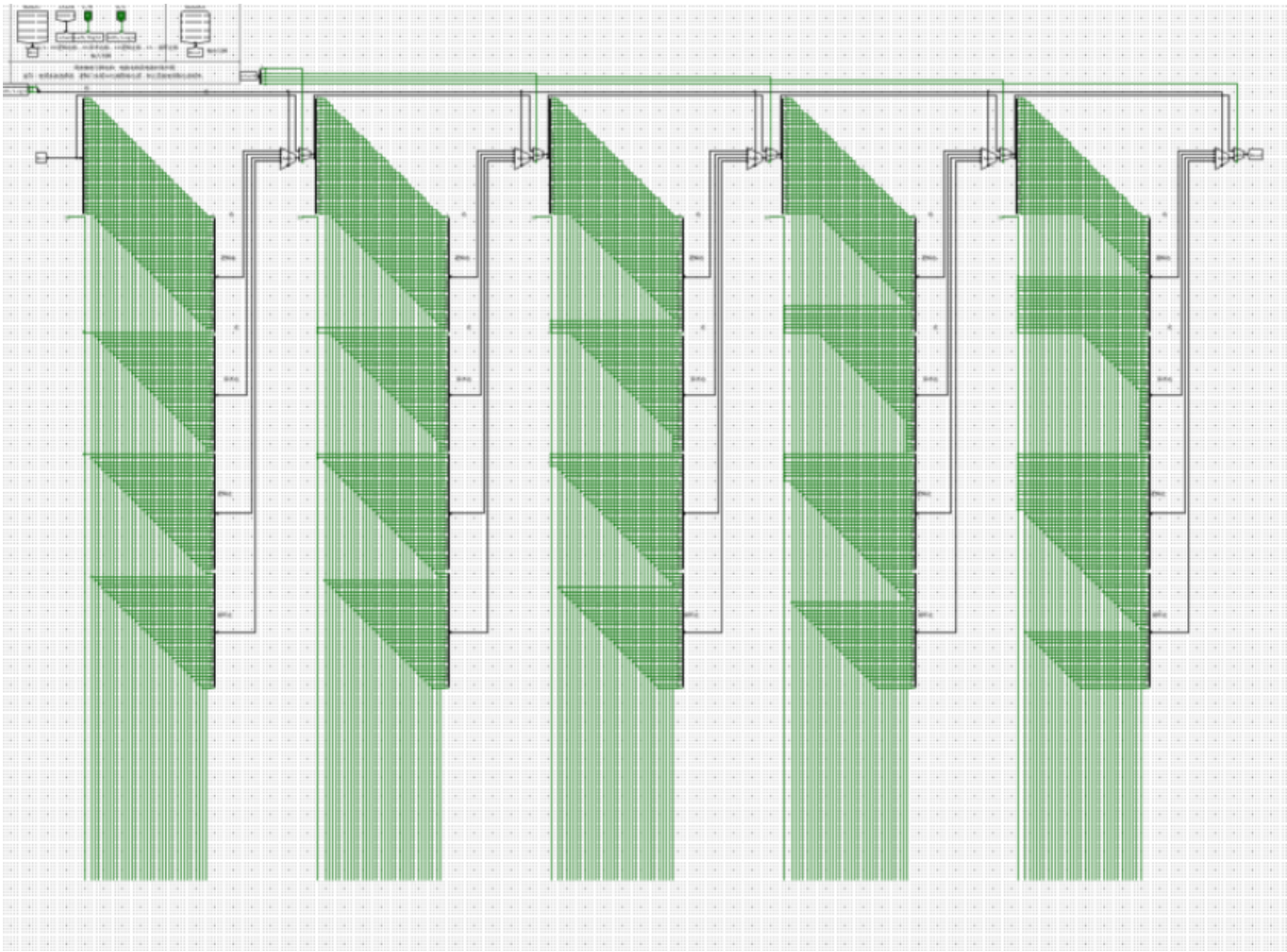


图4.3 实验电路图

（3）实验数据仿真测试图

> 要求：根据实验要求，输入测试数据，选择单步时钟执行，截取仿真运行时的电路图，分析电路状态是否满足设计需求。说明子模块的功能，列出子模块的功能表。

Cnt	Din	shamt	LR	AL	Dout
00	ffffffff	01	0	0	7fffffff
01	ffffffff	01	1	0	fffffffe
02	ffffffff	01	0	1	ffffffff
03	ffffffff	01	1	1	ffffffff
04	bbbbbbbb	07	0	0	01777777
05	bbbbbbbb	07	1	0	dddddd80
06	bbbbbbbb	07	0	1	ff777777
07	bbbbbbbb	07	1	1	dddddddd
08	aaaaaaaa	08	0	0	00aaaaaa
09	aaaaaaaa	08	1	0	aaaaaa00
0a	aaaaaaaa	08	0	1	ffaaaaaa
0b	eaaaaaaa	08	1	1	aaaaaaea
0c	aaaa5555	0b	0	0	0015554a
0d	aaaa5555	0b	1	0	52aaa800
0e	aaaa5555	0b	0	1	fff5554a
0f	aaaa5555	0b	1	1	52aaad55
10	77778888	11	0	0	00003bbb
11	77778888	11	1	0	11100000
12	77778888	11	0	1	00003bbb
13	77778888	11	1	1	1110eeef
14	55555aaa	15	0	0	000002aa
15	5555aaaa	15	1	0	55400000
16	5555aaaa	15	0	1	000002aa
17	5555aaaa	15	1	1	554aaab5
18	87654321	1c	0	0	00000008
19	87654321	1c	1	0	10000000
1a	87654321	1c	0	1	fffffff8
1b	87654321	1c	1	1	18765432
1c	deaddead	1f	0	0	00000001
1d	deaddead	1f	1	0	80000000
1e	deaddead	1f	0	1	ffffffff
1f	deaddead	1f	1	1	ef56ef56

Cnt	Din	shamt	LR	AL	Dout
00	ffffffff	01	0	0	7fffffff
01	ffffffff	01	1	0	fffffffe
02	ffffffff	01	0	1	ffffffff
03	ffffffff	01	1	1	ffffffff
04	bbbbbbbb	07	0	0	01777777
05	bbbbbbbb	07	1	0	dddddd80
06	bbbbbbbb	07	0	1	ff777777
07	bbbbbbbb	07	1	1	dddddddd
08	aaaaaaaa	08	0	0	00aaaaaa
09	aaaaaaaa	08	1	0	aaaaaa00
0a	aaaaaaaa	08	0	1	ffaaaaaa
0b	eaaaaaaa	08	1	1	aaaaaaea
0c	aaaa5555	0b	0	0	0015554a
0d	aaaa5555	0b	1	0	52aaa800
0e	aaaa5555	0b	0	1	fff5554a
0f	aaaa5555	0b	1	1	52aaad55
10	77778888	11	0	0	00003bbb
11	77778888	11	1	0	11100000
12	77778888	11	0	1	00003bbb
13	77778888	11	1	1	1110eeef
14	55555aaa	15	0	0	000002aa
15	5555aaaa	15	1	0	55400000
16	5555aaaa	15	0	1	000002aa
17	5555aaaa	15	1	1	554aaab5
18	87654321	1c	0	0	00000008
19	87654321	1c	1	0	10000000
1a	87654321	1c	0	1	fffffff8
1b	87654321	1c	1	1	18765432
1c	deaddead	1f	0	0	00000001
1d	deaddead	1f	1	0	80000000
1e	deaddead	1f	0	1	ffffffff
1f	deaddead	1f	1	1	ef56ef56

图4.4 实验仿真测试

（4）错误现象及分析

> 要求：在电路设计、连接和仿真运行时，遇到的任何错误，都需要截屏放置到实验报告中，并分析错误原因和解决办法。

在32位桶形移位器实验中，可能出现的错误主要集中在以下几个方面：首先，在扩展移位位数时，若未正确增加多路选择器的级数（如漏掉8位或16位移位对应的选择器层级），将导致高位数据移位失效。其次，在连接各级选择器时，若控制信号线序错乱（如将移位4位的控制信号误接到移位16位的选择器上），会造成移位量与实际控制不符。此外，在32位数据通路扩展过程中，若位扩展器连接不当（如高低位数据线交叉接反），会导致输出数据位序混乱。测试阶段需要特别注意边界情况验证，如全1数据循环移位时若未检查最高位和最低位的衔接，可能掩盖数据丢失错误。最后，封装时若输入输出端口定义不完整（如遗漏某条控制线），会影响后续电路集成使用。这些错误需要通过逐级验证移位功能和全位宽数据测试来排查。

5. RV32I 算术逻辑部件实验

（1）实验整体方案设计

> 要求：说明本次实验的顶层设计模块图，对每个子模块进行详细描述，定义输入输出引脚，数据及控制信号的传输通道等。

本实验基于RV32I指令集构建32位算术逻辑部件（ALU），作为CPU核心数据通路模块，集成加法器、移位器、逻辑运算单元及标志位生成电路，支持算术运算（加/减）、逻辑运算（与/或/异或）、移位操作（左/右、算术/逻辑）及带符号/无符号比较等7类运算。通过4位ALUctr控制信号（编码映射RV32I的funct3字段）生成5类子控制信号：SUBctr（加减选择）、SIGctr（符号比较模式）、ALctr（算术移位使能）、LRctr（移位方向）及3位OPctr（结果选择），采用最小项逻辑表达式（如SUBctr=∑m(2,3,8)）确保控制信号精确生成。实验分三步实现：1）设计ALUctrl控制部件，验证信号生成正确性；2）集成预制的32位CLA加法器与桶形移位器，通过多路选择器（7选1）按OPctr选择运算结果，其中标志位电路直接复用加法器的CF输出，Zero标志通过32位或非门检测；3）功能测试覆盖边界值（如0x80000000±0x7FFFFFFF）及所有ALUctr编码，验证算术右移的符号位扩展、无符号比较等关键功能。最终封装模块支持32位操作数A/B输入，输出Result和Zero标志，为后续CPU控制器提供标准化接口。

电路模块的封装情况示意图如下：

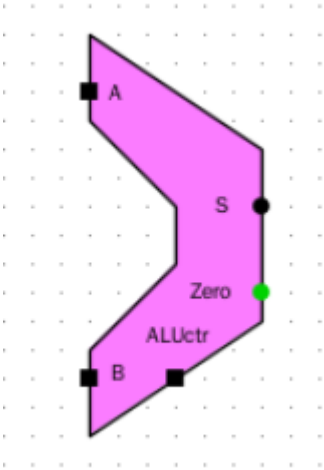


图5.1 电路封装示意图

（2）实验原理图和电路图

> 要求：给出每个子模块的原理图和 Logisim 中的电路图，定义子模块的外观图。如果对实验指导讲义中的内容提出优化或改进，需要此说明原因、方法和效果。

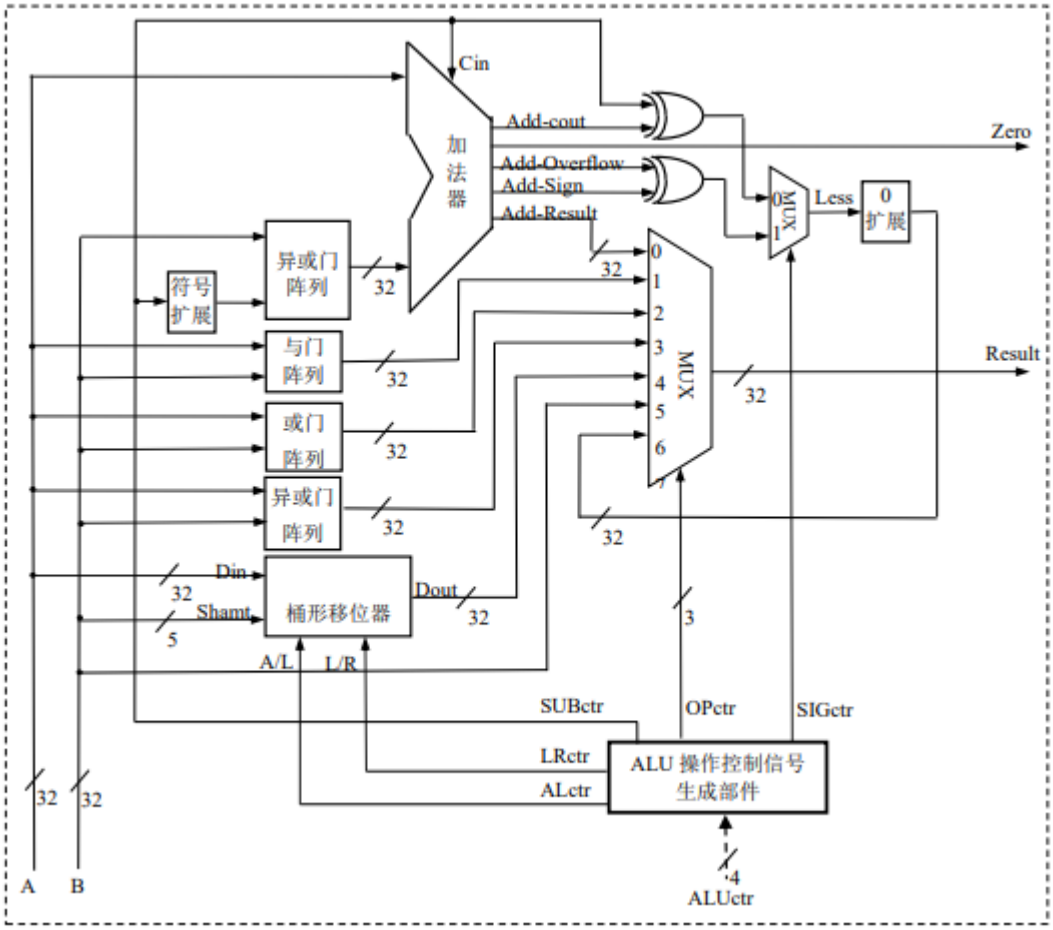


图5.2 实验原理图

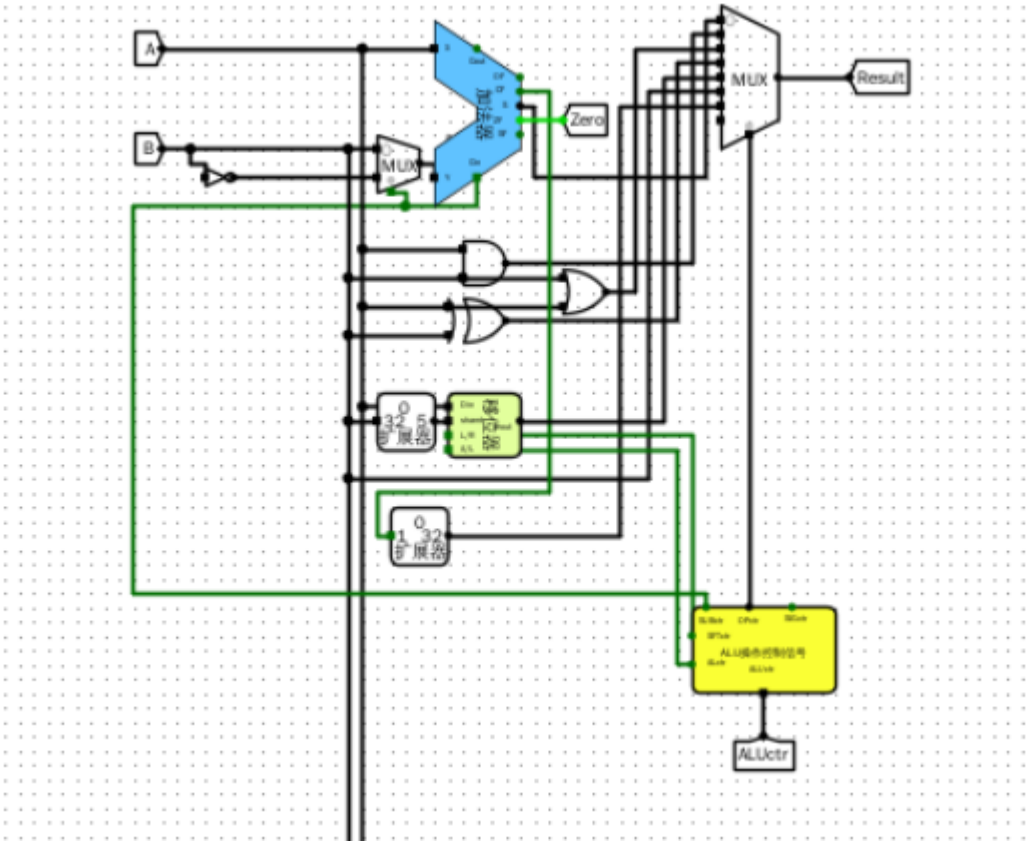


图5.3 实验电路图

（3）实验数据仿真测试图

> 要求：根据实验要求，输入测试数据，选择单步时钟执行，截取仿真运行时的电路图，分析电路状态是否满足设计需求。说明子模块的功能，列出子模块的功能表。

—— 预期输出 ——

Cnt	A	B	ALUctrResult		ZF
00	7fffffff	00000002	0	80000001	0
01	12345678	12345678	1	78000000	0
02	80000000	80000001	2	00000001	0
03	80000001	80000001	3	00000000	1
04	88887777	77778888	4	ffffffff	0
05	fedca987	fedcba98	5	000000fe	0
06	aaaa5555	5555aaaa	6	ffffffff	0
07	aaaa5555	5555aaaa	7	00000000	0
08	7fffffff	80000001	8	fffffffe	0
09	ffffffff	80000000	d	ffffffff	0
0a	12345678	a9876543	f	a9876543	0
0b	89abcded	89abcdef	3	00000001	0
0c	7fffffff	eeeffffff	8	91100000	0
0d	abcdabcd	fffbbbbb	8	abd1f012	0
0e	88997766	fff12345	0	888a9aab	0
0f	44553322	89674532	0	cdbc7854	0
10	fffffffe	00007fff	0	00007ffd	0
11	ffffdef0	87654321	1	ffffbde0	0
12	7fffffff	7fffffff	2	00000000	1
13	7ffffffe	fffffffe	3	00000001	0
14	77778888	ffffdddd	4	88885555	0
15	ffffeeee	aaaaa555	5	000007ff	0
16	bbbb4444	3333bbbb	6	bbbbffff	0
17	cccc3333	4444cccc	7	44440000	0
18	7effffff	00000002	8	7effffffd	0
19	7dffed89	0000000f	d	0000fbff	0
1a	ffff99ff	10000000	f	10000000	0
1b	ffffaaff	12345678	3	00000000	0
1c	7fffffff	7fffffff	8	00000000	1
1d	fffffffe	00000003	8	fffffffb	0

—— 实际输出 ——

展示原始输

Cnt	A	B	ALUctrResult		ZF
00	7fffffff	00000002	0	80000001	0
01	12345678	12345678	1	78000000	0
02	80000000	80000001	2	00000001	0
03	80000001	80000001	3	00000000	1
04	88887777	77778888	4	ffffffff	0
05	fedca987	fedcba98	5	000000fe	0
06	aaaa5555	5555aaaa	6	ffffffff	0
07	aaaa5555	5555aaaa	7	00000000	0
08	7fffffff	80000001	8	fffffffe	0
09	ffffffff	80000000	d	ffffffff	0
0a	12345678	a9876543	f	a9876543	0
0b	89abcded	89abcdef	3	00000001	0
0c	7fffffff	eeeffffff	8	91100000	0
0d	abcdabcd	fffbbbbb	8	abd1f012	0
0e	88997766	fff12345	0	888a9aab	0
0f	44553322	89674532	0	cdbc7854	0
10	fffffffe	00007fff	0	00007ffd	0
11	ffffdef0	87654321	1	ffffbde0	0
12	7fffffff	7fffffff	2	00000000	1
13	7ffffffe	fffffffe	3	00000001	0
14	77778888	ffffdddd	4	88885555	0
15	ffffeeee	aaaaa555	5	000007ff	0
16	bbbb4444	3333bbbb	6	bbbbffff	0
17	cccc3333	4444cccc	7	44440000	0
18	7effffff	00000002	8	7effffffd	0
19	7dffed89	0000000f	d	0000fbff	0
1a	ffff99ff	10000000	f	10000000	0
1b	ffffaaff	12345678	3	00000000	0
1c	7fffffff	7fffffff	8	00000000	1
1d	fffffffe	00000003	8	fffffffb	0

图5.4仿真测试情况

（4）错误现象及分析

> 要求：在电路设计、连接和仿真运行时，遇到的任何错误，都需要截屏放置到实验报告中，并分析错误原因和解决办法。

在RV32I算术逻辑部件(ALU)实验中，可能出现以下关键错误：首先，在ALU控制信号生成环节，若未严格按照最小项表达式实现SUBctr、SIGctr等控制信号（如将SUBctr=∑m(2,3,8)误实现为∑m(2,3)），会导致减法运算和比较指令执行异常。其次，在多路选择器连接时，若OPctr2 1 0位信号与功能选择对应关系错误（如将"与运算"的001误接到移位器输出端口），会造成运算结果选择混乱。此外，在标志位生成电路中，若Zero标志未正确检测全零结果（如仅检测低16位），会影响分支指令判断。测试阶段需特别注意边界情况，如算术右移时若未正确处理符号位扩展（ALctr=1），将导致负数移位结果错误。最后，封装时若未正确映射32位加法器的CF标志输出（如仍保留CIN异或电路），会造成进位标志冗余计算。这些错误需要通过逐条验证RV32I指令功能来排查，特别要检查带符号/无符号比较、算术/逻辑移位等关键功能的差异。

思考题在报告开头处